Below is a **master list of detailed Spring Boot terminology** (covering concepts used across Spring Boot **all major versions**), with **real-time use cases** and **examples**, in the form of a table.

✅ This includes terminology from: **Core Spring + Spring Boot + Spring MVC + Spring Data + Spring Security + Spring Cloud + Actuator + Testing + Packaging/Deployment**.

✅ Spring Boot Terminology (All Versions) — Detailed Master Table

Terminology	Meaning (What it is)	Real-time Use Case	Example (Practical)
Spring Boot	Framework to build Spring apps fast with minimal config	Build production-ready microservices quickly	Create REST APIs for ecommerce backend
Auto Configuration	Boot automatically configures beans based on classpath	Reduce XML/manual config	If spring-boot-starter-web exists → configures Tomcat + MVC
Starter	Predefined dependency bundle	Easy dependency management	spring-boot-starter-security adds security dependencies
spring-boot-starter-parent	Parent POM managing versions	Avoid dependency version conflicts	Common in Maven projects
Dependency Management	Controls versions of Spring libs automatically	Prevent jar mismatch	Fixes Spring + Jackson compatibility automatically
Embedded Server	Built-in server like Tomcat/Jetty/Undertow	Run app as a jar	java -jar app.jar
Fat Jar / Uber Jar	Jar containing app + dependencies	Easy deployment	Deploy single jar on Linux VM
WAR Packaging	Traditional packaging for external server	Legacy enterprise environments	Deploy to external Tomcat
SpringApplication	Boot entry point class	Start app programmatically	SpringApplication.run(App.class,args)

@SpringBootApplication	Combines @Configuration, @EnableAutoConfiguration, @ComponentScan	Main startup annotation	Used in root class
@EnableAutoConfiguration	Enables Boot auto config	Boot magic	Automatically creates DataSource bean
@ComponentScan	Scans packages for components	Auto-detect services/controllers	Finds @Service, @Repository
@Configuration	Defines configuration class	Central app config	Declares beans using @Bean
@Bean	Manually registers bean	Custom beans/config	Create RestTemplate bean
@Component	Generic Spring-managed component	Utility components	EmailValidatorService
@Service	Service layer component	Business logic	UserService.registerUser()
@Repository	DAO layer + exception translation	DB operations	UserRepository extends JpaRepository
@Controller	MVC Controller	Web pages/Thymeleaf	Return HTML page
@RestController	Controller + @ResponseBody	REST APIs	GET /users
@RequestMapping	Maps URL to controller	Base mappings	/api/users
@GetMapping	Maps HTTP GET	Fetch resources	GET /orders/{id}
@PostMapping	Maps HTTP POST	Create resources	POST /users
@PutMapping	Maps HTTP PUT	Update resource fully	PUT /users/{id}
@PatchMapping	Partial update	Update only some fields	Update user status only

@DeleteMapping	Deletes resource	Delete record	DELETE /users/{id}
@PathVariable	Reads URL variable	Fetch by id	/users/{id}
@RequestParam	Reads query param	Filtering/pagination	/users?page=1
@RequestBody	Reads JSON request body	Create/update entity	{ "name": "Anurag" }
@ResponseBody	Return response directly	REST response	Return JSON
DTO (Data Transfer Object)	Data object for API layer	Prevent exposing DB entity	UserResponseDTO
Entity	JPA table mapping class	ORM DB persistence	@Entity User
@Entity	Marks class as DB entity	Persistence	@Entity(name="users")
@Id	Primary key	Identify row	Long id
@GeneratedValue	Auto-generates primary key	Auto increment	GenerationType.IDENTITY
Repository Pattern	DB access abstraction	Clean code	UserRepository
CrudRepository	Basic CRUD repo	Simple DB operations	save(), findAll()
JpaRepository	Advanced JPA repository	Pagination + sorting	findAll(Pageable)
Pagination	Data returned in pages	large DB queries	PageRequest.of(0,20)
Sorting	Sort query results	ordered results	Sort by createdAt desc
Service Layer	Business rules layer	clean architecture	validation + DB calls
Controller Layer	API layer	inbound/outbound handling	request mapping + response

application.properties / yaml	Boot configuration files	env settings	DB url, port, logging
Profiles	environment-based config	dev/test/prod config	application-dev.yml
@Profile	Enable bean only in a profile	Prod-only beans	prod DB connection
Externalized Configuration	config outside code	secure deployments	use env vars for secrets
Spring Boot Config Binding	map props to classes	clean config handling	@ConfigurationProperties(prefix="app")
@Value	Inject property value	quick config injection	@Value("${server.port}")
@ConfigurationProperties	Strong typed config binding	best config pattern	AppProperties class
Environment	access system properties	flexible config read	env.getProperty("x")
Actuator	production monitoring endpoints	health + metrics	/actuator/health
Health Indicator	custom health checks	DB/Kafka health	checks Kafka connection
Metrics	app performance stats	monitoring dashboards	Prometheus metrics
Micrometer	metrics library integrated	observability	scrape CPU/heap/thread data
Logging	app logs	debugging and audits	log requests + errors
Log Levels	INFO/DEBUG/WARN/ERROR	control verbosity	debug only in dev
Security Filter Chain	filters for authentication	secure endpoints	JWT filter
Spring Security	security module	authentication/ authorization	OAuth2, JWT

JWT	stateless token-based auth	microservices auth	Authorization: Bearer token
OAuth2	third party identity login	login via Google	SSO support
RBAC	role-based access control	admin/user permissions	ADMIN can delete users
CORS	cross origin resource sharing	React frontend calls API	allow origin http://localhost:3000
CSRF	cross-site request forgery protection	security for browsers	disable in stateless APIs
Exception Handling	global error responses	consistent error handling	@ControllerAdvice
@ControllerAdvice	global exception handler	common exception mapping	handle validation errors
@ExceptionHandler	handle exception type	structured API error	return {error:"msg"}
Validation	input validation	prevent invalid data	@NotNull, @Email
@Valid	enable validation	auto validate DTO	@Valid @RequestBody
Bean Lifecycle	bean creation phases	resource mgmt	init + destroy hooks
@PostConstruct	init after injection	load cache	initialize config
@PreDestroy	cleanup before shutdown	close connections	close file handles
DI (Dependency Injection)	inject dependency automatically	loosely coupled code	inject repository into service
IoC Container	manages beans and DI	core Spring runtime	ApplicationContext
ApplicationContext	Spring container context	get bean registry	context.getBean()
Bean Scope	lifecycle scope	singleton/prototype	@Scope("prototype")

Singleton Scope	single instance bean	most services	default scope
Prototype Scope	new instance each time	stateful beans	custom builders
@RestTemplate	old HTTP client	call other APIs	internal service call
WebClient	reactive HTTP client	high-scale async calls	microservice gateway calls
Spring Cloud	tools for microservices	discovery/config/gateway	config server + Eureka
Eureka Client	service discovery	dynamic microservices	register service names
Config Server	central config repo	manage properties centrally	Git-based config
API Gateway	central routing layer	microservices routing	Spring Cloud Gateway
Circuit Breaker	prevent cascading failures	resilient architecture	Resilience4j circuit breaker
Rate Limiting	prevent abuse	public APIs	limit 100 requests/minute
Retry Mechanism	auto retry failures	transient error handling	retry DB down temporarily
Event Publishing	app event-based communication	decoupled modules	publish UserCreatedEvent
@EventListener	listen to events	background processing	send email on user creation
Scheduling	run tasks periodically	report generation	@Scheduled(cron=...)
@Scheduled	cron/fixedDelay tasks	job scheduling	daily backup tasks
@Async	async background execution	send email async	@Async sendMail()
Thread Pool Task Executor	manages async threads	optimize performance	custom executor for heavy load

Transaction	DB atomic operation	ensure consistency	money transfer
@Transactional	transaction boundary	commit/rollback	rollback when exception
Propagation	nested transaction behavior	microservice DB ops	REQUIRED/REQUIRES_NEW
Isolation	controls concurrency behavior	avoid dirty reads	SERIALIZABLE in banking
Spring Boot Test	testing support	integration tests	@SpringBootTest
@MockBean	mock beans in tests	isolate services	mock repo in test
@DataJpaTest	JPA testing slice	repository tests	test SQL operations
@WebMvcTest	controller slice testing	API tests	mock service and test controller
Integration Testing	full app test	real API + DB test	Testcontainers usage
Testcontainers	Docker-based test infra	real PostgreSQL in tests	ensures consistency
DevTools	auto restart + dev features	faster dev	hot reload in dev
LiveReload	auto reload browser	frontend dev	refresh browser automatically
Spring Boot CLI	run Groovy scripts quickly	rapid prototype	quick microservice demo
AOT Processing	ahead-of-time compilation	native fast startup	used in boot 3+
GraalVM Native Image	native compilation	serverless fast start	AWS Lambda APIs
Reactive Programming	async non-blocking approach	streaming systems	chat app, stock streaming
Mono / Flux	reactive types	reactive REST	Flux for streams
WebFlux	reactive web stack	high concurrency	100k+ concurrent connections

Tomcat	default embedded server	web apps	default in starter-web
Jetty	alternative server	low resource footprint	embedded Jetty
Undertow	high performance server	scalable apps	high throughput

---

If you want, I can also provide:

✅ **"300+ Spring Boot Terminology Master List"** (even deeper than above)
✅ Version-wise important additions: **Boot 1.x vs 2.x vs 3.x**
✅ A **real-world project architecture (layered microservice)** showing where each terminology is used.